

test python table

Table of contents

Setup	1
Markdown tables with python	1
Grid tables	3
Summary	4

Setup

Here, I will demonstrate how to generate markdown tables with python from within qmd files. We'll start by loading the packages we'll need:

```
import numpy as np
import pandas as pd
from IPython.display import Markdown, display
```

Additionally, you need the `tabulate` package. So, if you get an error that references this package, install it with:

```
pip install tabulate
```

Markdown tables with python

Next, we'll generate an example table with python and saving it to "out.csv". In most cases, tables will be manually created by users, but I do it with code for this example to demonstrate what data were included. For this table, I've intentionally given one of the cells a very long string value to demonstrate how line breaks are done automatically:

```

import pandas as pd

long_description = "This is a variable whose description is extremely long " \
    "and will make the table not fit on the entire page unless some formatting "\
    "trickery occurs, which pandas should be able to accommodate. I hope."

df = pd.DataFrame(
    columns = ["name", "description", "value"],
    data = [
        ["descriptive_integer", long_description, 15],
        ["concise_decimal", "This is a short description", 98.6],
        ["tiny_decimal", "A very small value", 0.0001],
        ["missing_value", "No value present", "NA"],
    ],
)

df.to_csv("out.csv", index=False)

```

Finally, we can demonstrate how to load this table using python. Specifically, we utilize `pandas.read_csv` to load the file as a `pandas.DataFrame`. Then we simply convert this data frame to a markdown table with the `.to_markdown` method. Then this markdown text is displayed with the `display` and `Markdown` functions, that process the text as markdown for quarto:

```

# read the table from a file
tab_data = pd.read_csv(
    "out.csv",
    dtype = {"value": str}, # force value to a string
    keep_default_na = False # keep missing values as written
)

# create markdown table
md_tab = tab_data.to_markdown(
    index = False, # don't include the index column
    colalign = ("left", "left", "right"), # alignment of the columns
    maxcolwidths = [None, 45, None], # limit 2nd column to 45 characters
)

# display the table
Markdown(md_tab)

```

Table 1: table loaded from csv file with the pandas python package

name	description	value
descriptive_integer	This is a variable whose description is extremely long and will make the table not fit on the entire page unless some formatting trickery occurs, which pandas should be able to accommodate. I hope.	15
concise_decimal	This is a short description	98.6
tiny_decimal	A very small value	0.0001
missing_value	No value present	NA

This functionality could be made even simpler by wrapping it all up in a function, very similar to what Geneva has started with `make_grid_table.py`. Then these code blocks simply include something like

```
# import this hypothetical function
from make_grid_table.py import make_markdown_table

# use the hypothetical function to display the table
make_markdown_table("out.csv", ...)
```

Grid tables

Grid tables can also be created in the same way (but for some reason the value column won't align properly in this specific example):

```
md_grid = tab_data.to_markdown(
    index = False,
    tablefmt = "grid",
    colalign = ("left", "left", "right"),
    maxcolwidths = [None, 45, None],
)

Markdown(md_grid)
```

name	description	value
descriptive_integer	This is a variable whose description is extremely long and will make the table not fit on the entire page unless some formatting trickery occurs, which pandas should be able to accommodate. I hope.	15
concise_decimal	This is a short description	98.6
tiny_decimal	A very small value	0.0001
missing_value	No value present	NA

I'm not sure why this is happening, since the actual text output looks correct:

```
print(md_grid)
```

```
+-----+-----+-----+
| name          | description                                     | value |
+=====+=====+=====+
| descriptive_integer | This is a variable whose description is      | 15 |
|                  | extremely long and will make the table not  |    |
|                  | fit on the entire page unless some formatting |    |
|                  | trickery occurs, which pandas should be able |    |
|                  | to accommodate. I hope.                     |    |
+-----+-----+-----+
| concise_decimal   | This is a short description                   | 98.6 |
+-----+-----+-----+
| tiny_decimal      | A very small value                           | 0.0001 |
+-----+-----+-----+
| missing_value     | No value present                             | NA |
+-----+-----+-----+
```

Summary

All told, this is a relatively straight-forward and flexible method of getting markdown tables in qmd files, since it allows users to simply create the tables in a spreadsheet type program, rather than manually typing them.

The `pandas.DataFrame.to_markdown()` leverages the power of `tabulate.tabulate` under the hood, and can accept all the same arguments and options. The arguments used above (`tablefmt`, `colalign`, and `maxcolwidths`) are all passed to `tabulate`.

Use the following code snippet to learn more about `tabulate`'s arguments:

```
from tabulate import tabulate
?tabulate
```